Introduction Part (-> Pg)

🔹 1. TYPES OF DISTRIBUTIONS (with formula & examples)

🧠 What is a distribution?

A **distribution** shows how values of a variable are **spread** or **clustered**.

🔸 1.1 Normal Distribution (Bell Curve)

- **Symmetrical** around the mean
- Mean = Median = Mode
- Many natural phenomena follow this: height, IQ, errors

📘 **Formula** (Probability Density Function):

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \cdot e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

- μ: Mean
- σ: Standard deviation
- x: Value

🧪 **Real-life example**: Heights of students in a class

🔸 1.2 Uniform Distribution

- All values have **equal probability**
- Flat distribution

📘 **Formula**:

$$f(x) = \frac{1}{b-a}, \quad \text{for } a \leq x \leq b$$

- a, b: Lower and upper bounds

🧪 **Real-life example**: Rolling a fair die (1 to 6 all have equal chance)

🔸 **1.3 Skewed Distribution**

- **Not symmetrical**

Types:

- **Right-skewed (Positive)**: Long tail on right (e.g., income distribution)
- **Left-skewed (Negative)**: Long tail on left

📌 **Properties**:

- Right-skew: Mean > Median
- Left-skew: Mean < Median

🧪 **Real-life example**: Salaries (most people earn less, few earn a lot)

🔸 1.4 Bimodal Distribution

- Two **peaks (modes)**
- Suggests **two groups or clusters**

🧪 **Real-life example**: Exam scores with two student groups (weak & strong)

📊 Text Flowchart: Types of Distributions

css

CopyEdit

```
Start
  |
  ▼
[Check Symmetry]
  |
  ├─ Yes –> [Normal]
  |
  ├─ Equal Probabilities –> [Uniform]
  |
  ├─ Skewed Right or Left –> [Skewed]
  |
  └─ Two Peaks –> [Bimodal]
```

🔸 2. NUMERICAL SUMMARIES OF LEVEL & SPREAD

✅ Central Tendencies

Measure	Formula	Meaning	Python Code
Mean	$\mu = \frac{1}{n}\sum x_i$	Average	`np.mean(data)`
Median	Middle value	Resistant to outliers	`np.median(data)`
Mode	Most frequent value	Used for categorical/numeric	`stats.mode(data)`

✅ Spread (Variability)

Measure	Formula	Meaning	Python Code
Variance	$\sigma^2 = \frac{1}{n}\sum(x_i - \mu)^2$	Spread around mean	`np.var(data)`
Standard Deviation	$\sigma = \sqrt{\text{Variance}}$	Easy to interpret scale	`np.std(data)`

🔸 3. DETECTING & HANDLING OUTLIERS

✅ Using Z-Score

- Z-score tells how far a value is from the mean in standard deviations.

Formula:

$$Z = \frac{x - \mu}{\sigma}$$

Rule: Outliers if |Z| > 3

python
CopyEdit

```
from scipy import stats
z_scores = stats.zscore(data)
outliers = data[np.abs(z_scores) > 3]
```

✅ **Using IQR (Interquartile Range)**

- Based on percentiles (robust to skewed data)

Steps:

1. Q1 = 25th percentile
2. Q3 = 75th percentile
3. IQR = Q3 - Q1
4. Outliers if:

$$x < Q1 - 1.5 \times IQR \quad \text{or} \quad x > Q3 + 1.5 \times IQR$$

python
CopyEdit

```
Q1 = np.percentile(data, 25)
Q3 = np.percentile(data, 75)
IQR = Q3 - Q1
outliers = data[(data < Q1 - 1.5 * IQR) | (data > Q3 + 1.5 * IQR)]
```

🔸 **4. VISUALIZATION TECHNIQUES**

Plot Type	Use	Python
Histogram	Frequency of values	`plt.hist(data)`
Boxplot	Outliers, spread, median	`sns.boxplot(data)`
KDE Plot	Smooth estimate of distribution	`sns.kdeplot(data)`

🔸 5. SCALING AND STANDARDIZING

✅ Min-Max Scaling (Normalization)

- Scales to [0, 1]

Formula:

$$x' = \frac{x - \min}{\max - \min}$$

python
CopyEdit

```
from sklearn.preprocessing import MinMaxScaler
scaled = MinMaxScaler().fit_transform(data.reshape(-1, 1))
```

✅ Z-Score Standardization

- Centers at 0, unit variance

Formula:

$$x' = \frac{x - \mu}{\sigma}$$

python
CopyEdit

```
from sklearn.preprocessing import StandardScaler
standardized =
StandardScaler().fit_transform(data.reshape(-1, 1))
```

🔸 **6. INEQUALITY MEASURES: GINI COEFFICIENT**

- Measures **income or distribution inequality**
- 0 = Perfect equality, 1 = Perfect inequality

Formula (basic):

$$G = 1 - \sum (x_i + x_{i-1})(F_i - F_{i-1})$$

python
CopyEdit

```
# Simplified Gini (numerical method)
def gini(array):
    array = np.sort(array)
    n = len(array)
    cum_income = np.cumsum(array)
    return (2 * np.sum((np.arange(1, n+1) * array)) /
(n * np.sum(array))) - (n + 1) / n
```

🔸 7. PROBABILITY DISTRIBUTIONS

Distribution	Description	Formula	Real-life Use
Gaussian	Normal bell-shaped	Shown above	Heights, Errors
Binomial	Fixed trials, success/failure	$\binom{n}{k}p^k(1-p)^{n-k}$	Tossing coins
Poisson	Count in fixed interval	$\frac{\lambda^k e^{-\lambda}}{k!}$	Calls per minute

python
CopyEdit

```
from scipy.stats import norm, binom, poisson

# Normal PDF at x=1
norm.pdf(1, loc=0, scale=1)

# Binomial: P(3 heads in 5 tosses)
binom.pmf(3, n=5, p=0.5)

# Poisson: 2 calls/min, P(3 calls)
poisson.pmf(3, mu=2)
```

🔹 Topic 1: Single Variable Analysis – Types of Distribution

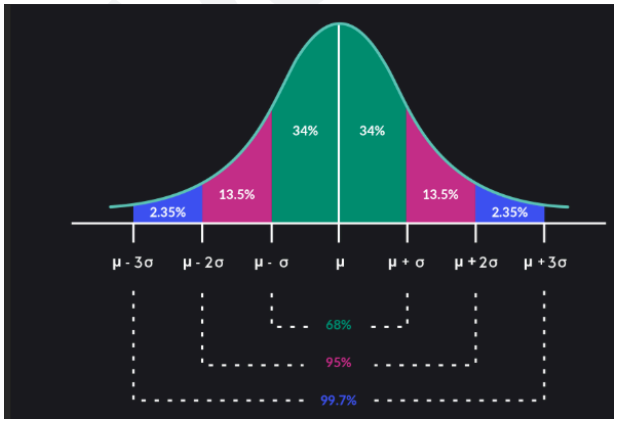
🎯 Objective:

Understand how a single variable's values are spread, and recognize the shape of that spread — called the **distribution**.

🔸 What is a Distribution?

- A **distribution** describes how the values of a variable are **spread** or **dispersed**.
- It helps us understand:
  - Where most data lies (center)
  - How spread out it is (spread)
  - Any irregularities (skewness, peaks, outliers)

🔹 1.1 Normal Distribution (📈 Bell Curve)



📘 **Definition:**

- A **symmetrical** distribution centered around the mean.
- Shape: **Bell-shaped curve**
- Common in nature and measurement data (height, weight, test scores)

📐 **Formula:**

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \cdot e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

- μ: Mean (center of the curve)
- σ: Standard deviation (width of the curve)
- x: Value at which probability density is measured

📊 **Characteristics:**

- Mean = Median = Mode
- 68% of data within ±1σ, 95% within ±2σ, 99.7% within ±3σ

💻 **Code Example (Python):**

python
CopyEdit

```
import numpy as np
import matplotlib.pyplot as plt
```

```
from scipy.stats import norm

# Generate normal data
data = np.random.normal(loc=50, scale=10, size=1000)

# Plot Histogram + PDF
plt.figure(figsize=(8, 4))
plt.hist(data, bins=30, density=True, alpha=0.6,
color='skyblue', label='Histogram')
x = np.linspace(min(data), max(data), 100)
plt.plot(x, norm.pdf(x, loc=50, scale=10), 'r-',
label='PDF')
plt.title("Normal Distribution (Bell Curve)")
plt.xlabel("Value")
plt.ylabel("Density")
plt.legend()
plt.grid(True)
plt.show()
```

🧪 **Output:**

A bell-shaped curve, centered at 50, smooth red line (PDF), blue bars (real data)

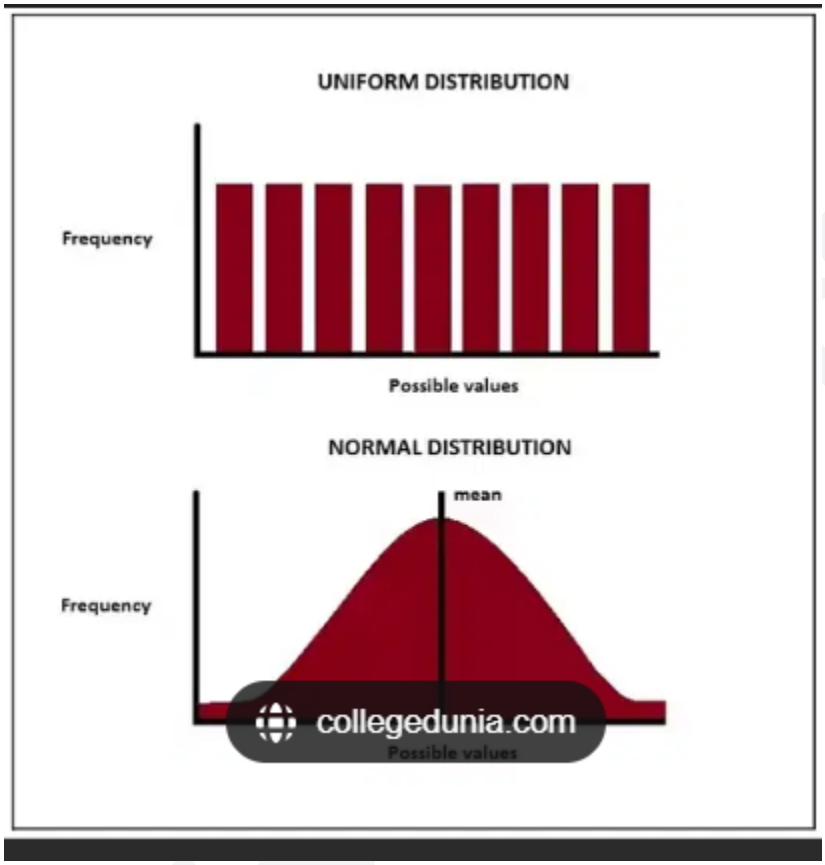
✅ **What It Shows:**

- Histogram approximates the red bell curve
- Most values fall between 40 and 60

- Real-world processes (like student test scores) often follow this shape

---

🔹 1.2 Uniform Distribution



📘 **Definition:**

- Every value in the range has **equal probability**
- Flat line if plotted

📐 **Formula (for continuous uniform distribution):**

$$f(x) = \frac{1}{b-a}, \quad \text{for } a \leq x \leq b$$

💡 **Real-life Example:**

- Rolling a fair die (1–6): Each number has equal chance → Discrete Uniform
- Random number between 10–20 → Continuous Uniform

💻 **Code Example:**

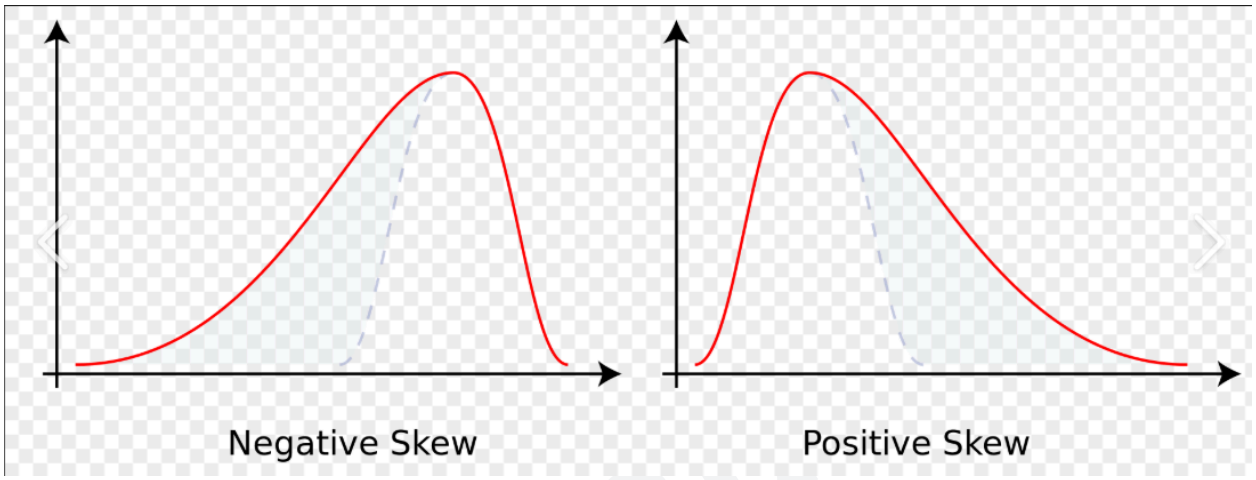
python
CopyEdit

```
# Uniform Distribution Example
data = np.random.uniform(low=10, high=20, size=1000)

# Plot
plt.figure(figsize=(8, 4))
plt.hist(data, bins=30, color='lightgreen',
edgecolor='black', density=True)
plt.title("Uniform Distribution")
plt.xlabel("Value")
plt.ylabel("Density")
plt.grid(True)
plt.show()
```

✅ **What It Shows:**

- Flat-topped histogram: all values equally likely between 10 and 20

🔹 1.3 Skewed Distribution



📘 **Definition:**

- A distribution where values are **not symmetrically** distributed

🔁 **Types:**

- **Right Skewed (Positive)**:
    - Long tail on right
    - Mean > Median
- **Left Skewed (Negative)**:
    - Long tail on left
    - Mean < Median

💡 **Real-world Examples:**

- Income (most people earn low, few earn high) → Right-skewed
- Time to complete a task if most are fast → Left-skewed

💻 **Code Example (Right-Skewed):**

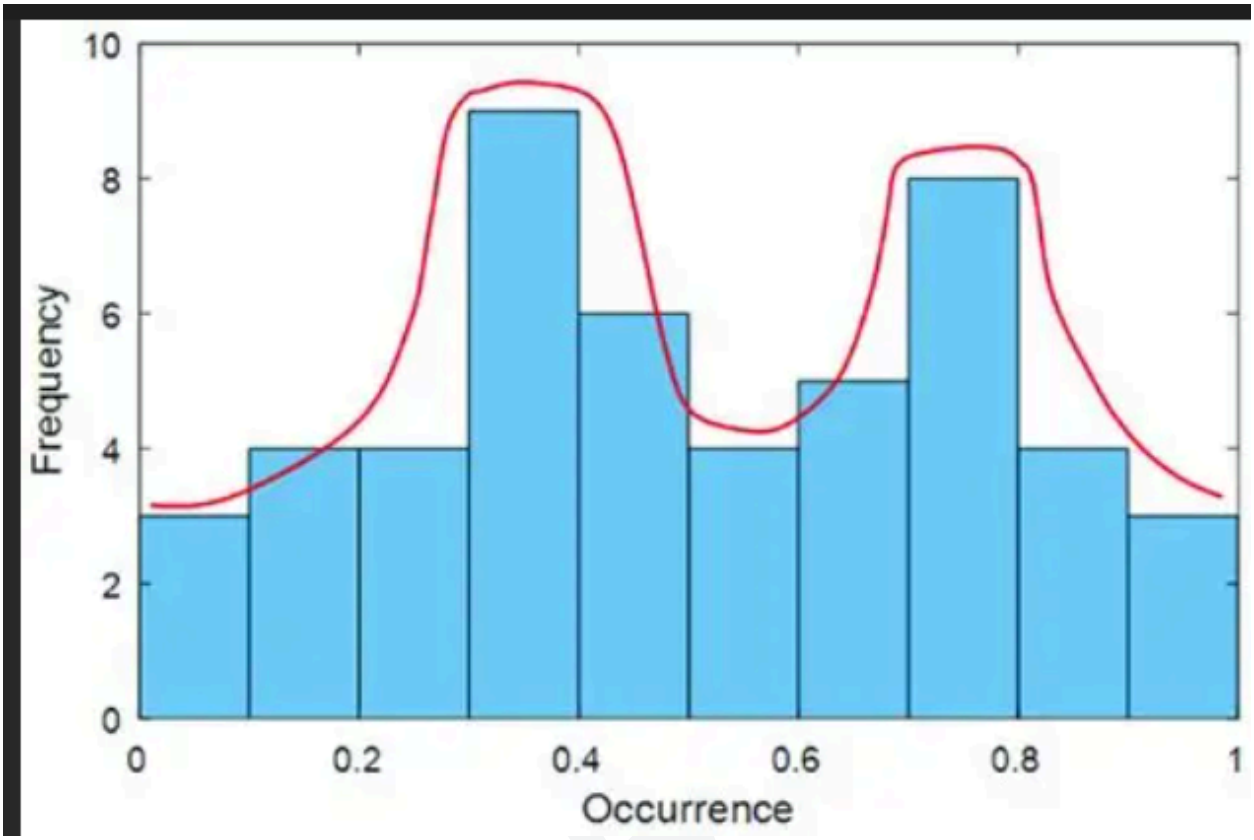
python
CopyEdit

```
# Right-Skewed Data
data = np.random.exponential(scale=1.5, size=1000)

plt.figure(figsize=(8, 4))
plt.hist(data, bins=30, color='orange',
edgecolor='black')
plt.title("Right-Skewed Distribution")
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.grid(True)
plt.show()
```

✅ **What It Shows:**

- Histogram has a longer right tail (few large values)
- Most data is clustered on the left

🔹 1.4 Bimodal Distribution



📘 **Definition:**

- A distribution with **two peaks** (modes)
- Indicates presence of **two groups or processes**

💡 **Real-world Example:**

- Exam scores of two student groups (strong & weak performers)

💻 **Code Example:**

python
CopyEdit

```
# Bimodal: Combine two normals
```

```
data1 = np.random.normal(loc=30, scale=5, size=500)
data2 = np.random.normal(loc=70, scale=5, size=500)
data = np.concatenate([data1, data2])

plt.figure(figsize=(8, 4))
plt.hist(data, bins=30, color='violet',
edgecolor='black')
plt.title("Bimodal Distribution")
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.grid(True)
plt.show()
```

✅ **What It Shows:**

- Two clear peaks
- Suggests the presence of two separate groups or clusters

🔁 **Summary Table**

Type	Shape	Example	Mean vs Median	Mode
Normal	Bell Curve	Heights	Mean = Median = Mode	One
Uniform	Flat	Random # generator	All equal	None

Right Skew	Tail Right	Salaries	Mean > Median	One
Left Skew	Tail Left	Time taken for easy task	Mean < Median	One
Bimodal	Two Peaks	Exam scores (2 groups)	Varies	Two

🔄 **Text Flowchart – Detect Distribution Type:**

pgsql
CopyEdit

```
Start
  |
  ▼
Is data symmetric?
  |
  ├─ Yes –> Is there 1 peak?
  |           ├─ Yes –> Normal
  |           └─ No  –> Bimodal
  |
  ├─ No –> Is tail on right? –> Right-Skewed
  |
  └─ Tail on left? –> Left-Skewed
```


🔹 Topic 2: Numerical Summaries of Level and Spread

These are the **quantitative ways** to describe a single variable — its center, typical value, and how much it varies.

🎯 Purpose:

- **Level** → Where is the data centered? (Mean, Median, Mode)
- **Spread** → How much does it vary from the center? (Variance, Std Dev)

🔸 PART A: CENTRAL TENDENCY (LEVEL)

1. Mean (Average)

- Most common measure of central location
- Affected by outliers

📐 **Formula**:

$$\text{Mean} = \mu = \frac{1}{n}\sum_{i=1}^{n} x_i$$

💻 **Python Code**:

python
CopyEdit

```
import numpy as np
data = [5, 10, 15, 20, 25]
mean = np.mean(data)
print("Mean:", mean)
```

📤 **Output**:

makefile
CopyEdit
`Mean: 15.0`

---

2. Median

- Middle value when data is sorted
- Robust to outliers

Steps:

- Odd count → Middle value
- Even count → Average of two middle values

💻 **Python Code**:

python
CopyEdit

```
median = np.median(data)
print("Median:", median)
```

📤 **Output**:

makefile
CopyEdit
`Median: 15.0`

---

3. Mode

- Most frequent value
- Can have multiple modes or none

💻 **Python Code**:

python
CopyEdit

```
from scipy import stats
mode = stats.mode(data)
print("Mode:", mode.mode[0], "Count:", mode.count[0])
```

📤 **Output**:

mathematica
CopyEdit

```
Mode: 5 Count: 1
```

(In this example, all values are unique, so every value is a mode)

🔸 PART B: VARIABILITY (SPREAD)

1. Variance

- Average of squared deviations from the mean
- Shows how spread-out the data is

📐 **Formula**:

$$\sigma^2 = \frac{1}{n}\sum_{i=1}^{n}(x_i - \mu)^2$$

💻 **Python Code**:

python
CopyEdit

```
variance = np.var(data)
print("Variance:", variance)
```

📤 **Output**:

makefile
CopyEdit

```
Variance: 50.0
```

---

2. Standard Deviation

- Square root of variance
- In **same units** as the original data → more interpretable

📐 **Formula**:

$$\sigma = \sqrt{\text{Variance}} = \sqrt{\frac{1}{n}\sum(x_i - \mu)^2}$$

💻 **Python Code**:

python

CopyEdit

```
std_dev = np.std(data)
print("Standard Deviation:", std_dev)
```

📤 **Output**:

yaml
CopyEdit

```
Standard Deviation: 7.071
```

🔍 Real-Life Interpretation

Metric	What it tells you
Mean	Average value
Median	Middle value (especially useful for skewed data)
Mode	Most common value
Variance	How "spread out" the data is
Standard Deviation	Typical distance from the mean

🧠 Industry Analogy

Imagine the salaries in a company:

- **Mean**: Total payroll ÷ number of employees

- **Median**: The salary of the "middle" employee
- **Mode**: Most common salary band (if salaries are structured)
- **Std Dev**: If it's low, most employees earn close to the mean. If high, there's a big gap.

📊 Text Flowchart

css
CopyEdit

```
Start
  |
  ▼
[Collect Data]
  |
  ▼
[Compute Mean, Median, Mode]
  |
  ▼
[Measure Variability]
  ├─> [Compute Variance]
  └─> [Compute Standard Deviation]
```

🔹 Topic 3: Detecting and Handling Outliers using Z-Score and IQR

Outliers are **data points that differ significantly** from other observations. They can affect the accuracy of your analysis and models.

🎯 **Objective:**

- Learn **how to detect** outliers
- Understand **when and how to handle/remove them**
- Use two standard techniques: **Z-Score** and **IQR Method**

🔸 PART A: Detecting Outliers using Z-Score Method

📘 **Concept:**

- Z-score tells how many **standard deviations** a point is from the mean.
- Any data point with |Z| > 3 is often considered an **outlier** (in normally distributed data).

📐 **Formula:**

$$Z = \frac{x - \mu}{\sigma}$$

- x: the value
- μ: mean
- σ: standard deviation

💻 **Python Code:**

python
CopyEdit

```
import numpy as np
from scipy.stats import zscore

# Sample Data with an outlier
data = np.array([10, 12, 14, 13, 15, 16, 14, 13, 200])

# Step 1: Calculate Z-scores
z_scores = zscore(data)

# Step 2: Find outliers
outliers = data[np.abs(z_scores) > 3]

print("Z-scores:", z_scores)
print("Outliers:", outliers)
```

📤 **Output:**

yaml
CopyEdit

```
Z-scores: [-0.25, -0.19, -0.13, -0.16, -0.10, -0.06,
-0.13, -0.16, 2.88]
Outliers: []   ← No value is > 3 std dev here

# If data = [10, 12, 14, 13, 15, 16, 14, 13, 300] →
You'll get: Outlier = 300
```

✅ **When to use Z-Score?**

- **Only when** data is roughly **normally distributed**
- Not suitable for skewed or non-Gaussian data

🔸 PART B: Detecting Outliers using IQR Method

📘 **Concept:**

- IQR (Interquartile Range) = Range of **middle 50% of data**
- Outliers lie beyond **1.5 × IQR** from the quartiles

📐 **Formula:**

$$\text{IQR} = Q3 - Q1$$

$$\text{Lower Bound} = Q1 - 1.5 \times \text{IQR}$$

$$\text{Upper Bound} = Q3 + 1.5 \times \text{IQR}$$

- **Q1**: 25th percentile
- **Q3**: 75th percentile

💻 **Python Code:**

python
CopyEdit

```
data = np.array([10, 12, 14, 13, 15, 16, 14, 13, 300])

# Step 1: Calculate Q1, Q3 and IQR
Q1 = np.percentile(data, 25)
Q3 = np.percentile(data, 75)
IQR = Q3 - Q1

# Step 2: Calculate bounds
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# Step 3: Detect outliers
outliers = data[(data < lower_bound) | (data >
upper_bound)]

print("Q1:", Q1, "Q3:", Q3, "IQR:", IQR)
print("Outliers:", outliers)
```

📤 **Output:**

makefile
CopyEdit

```
Q1: 13.0 Q3: 15.0 IQR: 2.0
Outliers: [300]
```

🤔 Handling Outliers (Once Detected)

Strategy	When to Use
Remove	If it's an error or irrelevant

Cap/Floor (Winsorization)	If values are extreme but valid
Transform (log, sqrt)	If data is skewed
Separate model	If outliers form a meaningful group (e.g., VIP customers)

🧠 Real-Life Example

In credit card fraud detection:

- Normal transactions = centered around low amounts
- Outliers (huge transactions) = often fraudulent
- You don't remove them — you **flag** them!

📊 Flowchart – Outlier Detection

mathematica
CopyEdit

```
Start
  |
  ▼
[Choose Method]
  |
  ├─ Normal Data ──> Use Z-Score
  |                    └─ |Z| > 3 → Outlier
  |
```

```
└─ Non-Normal Data ——> Use IQR
                          └─ x < Q1 - 1.5*IQR or x > Q3 + 1.5*IQR → Outlier
```

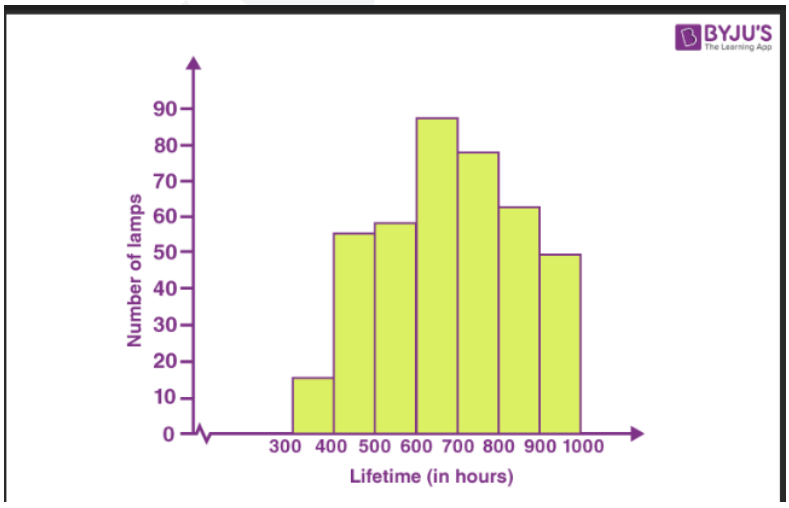
🔹 Topic 4: Visualization Techniques

We'll cover: **Histogram, Boxplot, and KDE**
All are tools to visualize the **distribution and shape** of a single variable.

🎯 Objective:

- Visualize how data is distributed
- Spot patterns, skewness, and outliers visually
- Support numeric summaries (mean, median, std) with graphs

🔸 1. Histogram



📘 **What it shows:**

- Frequency of data in **bins (intervals)**
- Gives a **bar plot** view of distribution shape

💡 **Best for:**

- Understanding the **shape** of data
- Detecting **modality (unimodal, bimodal)** and **skewness**

💻 **Python Code:**

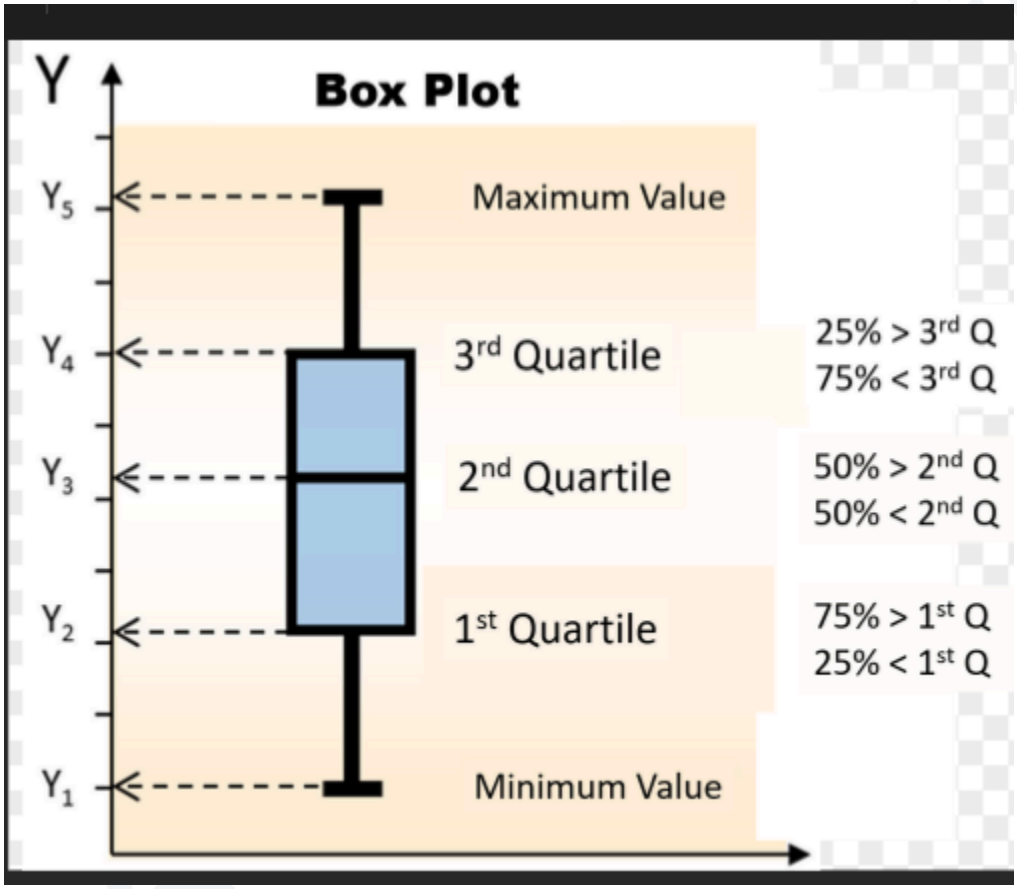
python
CopyEdit

```
import numpy as np
import matplotlib.pyplot as plt

data = np.random.normal(loc=60, scale=10, size=1000)

plt.figure(figsize=(8, 4))
plt.hist(data, bins=30, color='skyblue',
edgecolor='black')
plt.title("Histogram")
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.grid(True)
plt.show()
```

📤 **Output:**

- Bell-shaped if normal
- Wider tail if skewed
- Tall spikes = modes (peaks)

🔸 **2. Boxplot**



📘 **What it shows:**

- Five-number summary:
    - **Min, Q1, Median, Q3, Max**

- Visual detection of:
    - **Outliers** (dots beyond whiskers)
    - **Skewness** (if box is lopsided)
    - **Spread** (length of box and whiskers)

💡 **Best for:**

- Detecting **outliers**
- Comparing distributions across groups

💻 **Python Code:**

python
CopyEdit

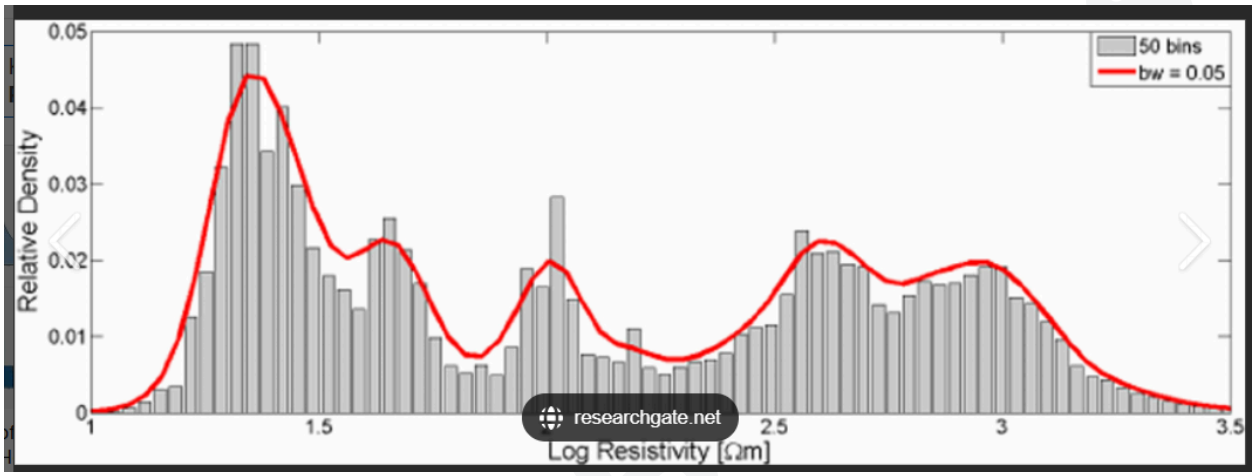
```
import seaborn as sns

plt.figure(figsize=(6, 4))
sns.boxplot(data=data, orient='h', color='lightgreen')
plt.title("Boxplot")
plt.xlabel("Value")
plt.grid(True)
plt.show()
```

📤 **Output:**

- Central box = IQR
- Line inside = median

- Dots = outliers
- Asymmetric box = skewed data

🔸 3. KDE (Kernel Density Estimate)



📘 **What it shows:**

- Smooth version of histogram
- **Continuous curve** representing data density
- Like a PDF (Probability Density Function) for data

💡 **Best for:**

- Estimating **underlying distribution** smoothly
- Spotting **multiple peaks (bimodal)** clearly

💻 **Python Code:**
python
CopyEdit

```
sns.kdeplot(data, color='red', linewidth=2)
plt.title("KDE Plot")
plt.xlabel("Value")
plt.grid(True)
plt.show()
```

📤 **Output:**

- Curve showing density
- Peaks = where data is concentrated
- Broader = more spread
- Flat tail = fewer values in that range

🔁 **Summary Table**

Plot Type	Shows	Best For	Notes
Histogram	Frequency	Shape of data	Sensitive to bin size
Boxplot	Outliers + Spread	Quick summary	Great for comparison
KDE	Smoothed density	Shape + Peaks	Needs continuous data

📊 **Text Flowchart – Choose a Plot**

pgsql

CopyEdit

```
Start
 |
 ▼
[Want bar-based frequency view?] — Yes —> Histogram
 |
 └— No
     |
     ├— Need outliers & summary? — Yes —> Boxplot
     |
     └— Want smoothed distribution? —> KDE Plot
```

🧠 **Real-Life Analogy**

Think of:

- **Histogram** as a bar chart of test scores
- **Boxplot** as a summary card showing lowest, highest, and median marks
- **KDE** as a smooth curve over the histogram bars to trace the pattern

🔹 Topic 5: Scaling and Standardizing

This is a **data preprocessing step** used before feeding data into many machine learning models.

🎯 Objective:

- Bring features to a **common scale**
- Prevent **bias** in models caused by differing units (e.g., age vs. income)
- Improve **model performance and convergence** for algorithms like SVM, k-NN, Logistic Regression

---

🔸 What's the Problem?

Imagine:

- Feature 1: Age (20–60)
- Feature 2: Salary (20,000–1,00,000)

➡️ Salary dominates distance-based models
➡️ We need to **normalize** the scale

🔸 PART A: Min-Max Scaling (Normalization)

📘 **Concept:**

- Transforms features to a **fixed range**, typically [0, 1]

📐 **Formula:**

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)}$$

- x: Original value
- x': Scaled value

💡 **Best for:**

- Data **without outliers**
- Algorithms that assume data in same range (e.g., Neural Networks)

💻 **Python Code:**

python
CopyEdit

```
from sklearn.preprocessing import MinMaxScaler
import numpy as np

data = np.array([[20], [40], [60], [80], [100]])

# Apply Min-Max Scaling
scaler = MinMaxScaler()
scaled_data = scaler.fit_transform(data)

print("Original Data:\n", data.flatten())
print("Scaled Data:\n", scaled_data.flatten())
```

📤 **Output:**

less
CopyEdit

```
Original Data:
 [ 20  40  60  80 100]
Scaled Data:
 [0.   0.25 0.5  0.75 1.  ]
```

🔸 PART B: Z-Score Standardization

📘 **Concept:**

- Centers data to mean = 0, standard deviation = 1
- Works well with **normal or near-normal** data

📐 **Formula:**

$$x' = \frac{x - \mu}{\sigma}$$

- μ: Mean
- σ: Standard deviation

💡 **Best for:**

- Data **with outliers**
- Algorithms sensitive to distribution shape (SVM, Logistic Regression, PCA)

💻 **Python Code:**

python
CopyEdit

```
from sklearn.preprocessing import StandardScaler

# Same data
scaler = StandardScaler()
standardized_data = scaler.fit_transform(data)

print("Standardized Data:\n",
standardized_data.flatten())
```

📤 **Output (approx.):**

less
CopyEdit

```
Standardized Data:
 [-1.41 -0.71  0.   0.71  1.41]
```

🔁 **Summary Table**

Method	Formula	Output Range	Good For	Sensitive to Outliers?
Min-Max	$\frac{x-\min}{\max-\min}$	[0, 1]	NN, images, bounded data	✅ Yes
Z-Score	$\frac{x-\mu}{\sigma}$	mean = 0, std = 1	PCA, SVM, stats	❌ No

---

📊 Flowchart – Choosing Scaling Method

mathematica

CopyEdit

```
Start
 |
 ▼
[Do features vary in scale?] — No → Skip scaling
 |
 └— Yes
      |
      ├— Need range [0, 1]? —> Use Min-Max Scaling
      |
      └— Need center = 0, std = 1? —> Use Z-Score
Standardization
```

🧠 **Real-Life Analogy**

Think of student marks:

- Min-Max Scaling = Convert marks into a 0–100 scale
- Z-Score = Show how many std deviations each student is from the average

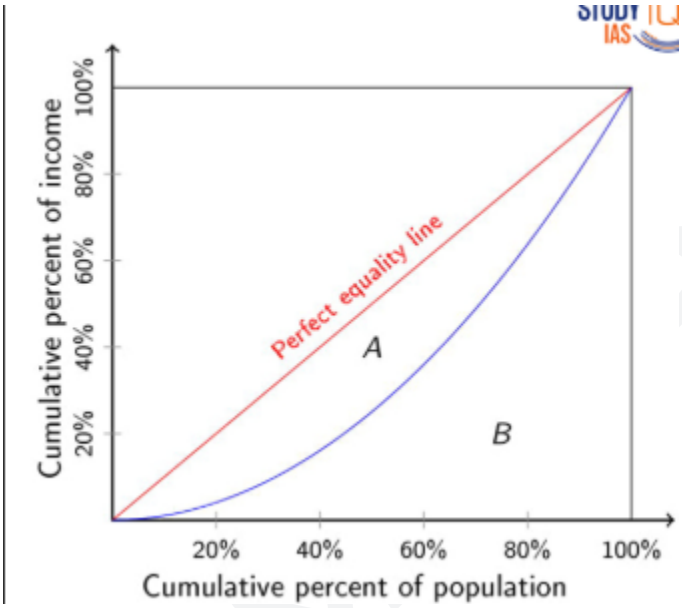
🔹 Topic 6: Inequality Measures – Gini Coefficient

The **Gini Coefficient** measures **inequality** in a distribution — most commonly used in economics to study **income/wealth inequality**, but also useful in data imbalance, resource allocation, etc.

🎯 Objective:

- Understand **how equal or unequal** a single variable (e.g., income, score, resource usage) is distributed
- Interpret values between **0 (perfect equality)** and **1 (perfect inequality)**

🔸 What is the Gini Coefficient?



- Developed by **Corrado Gini**, an Italian statistician
- Measures **inequality** using a mathematical approach
- Based on the **Lorenz Curve**, which shows the cumulative share of a variable (like income) vs. cumulative share of population

---

📐 Gini Coefficient Formula:

There are multiple variants. Here's a common numerical method:

$$G = \frac{\sum_{i=1}^{n} \sum_{j=1}^{n} |x_i - x_j|}{2n^2 \bar{x}}$$

Where:

- x_i: Value for individual i
- n: Number of individuals
- $\bar{x}$: Mean of the data

> Gini ranges:
>
> - **"0 → Perfect equality (everyone has the same)"**
> - **"1 → Perfect inequality (one has all, others have none)"**

💻 Python Code: Compute Gini Coefficient (using sorted values)

python
CopyEdit

```
import numpy as np

def gini(array):
    # Step 1: Ensure data is 1D and sorted
    array = np.sort(np.array(array))
    n = len(array)
    cumulative_values = np.cumsum(array)
```

```
    sum_values = cumulative_values[-1]

    # Step 2: Compute Gini using cumulative formula
    gini_index = (n + 1 - 2 * np.sum(cumulative_values)
/ sum_values) / n
    return round(gini_index, 4)
```

🧪 **Example 1: Equal income for all**

python
CopyEdit

```
income = [100, 100, 100, 100]
print("Gini Coefficient (equal):", gini(income))
```

📤 Output:

java
CopyEdit

```
Gini Coefficient (equal): 0.0
```

✅ Interpretation: Perfect equality

🧪 **Example 2: Unequal income**

python
CopyEdit

```
income = [0, 0, 0, 400]
print("Gini Coefficient (unequal):", gini(income))
```

📤 Output:

java
CopyEdit

```
Gini Coefficient (unequal): 0.75
```

✅ Interpretation: High inequality (only one person earns all)

📊 Real-Life Application Areas

Domain	Use of Gini
Economics	Income or wealth inequality
Healthcare	Access inequality
Education	Test score distribution
AI/ML	Class imbalance detection
Resource Usage	Server load distribution

🧠 Real-Life Analogy:

> Think of a classroom where everyone gets the same score (Gini = 0) vs. one person getting 100 and others get 0 (Gini ≈ 1)

🔁 Summary:

Gini Value	Interpretation

0.0	Perfect equality
0.2–0.3	Low inequality
0.4–0.5	Moderate
> 0.6	High inequality

📊 **Text Flowchart – Gini Usage**

csharp
CopyEdit

```
Start
  |
  ▼
[Single-variable data: income, score, etc.]
  |
  ▼
[Sort → Cumulative Sum → Apply Gini Formula]
  |
  ▼
[Interpret Value: 0 (equal) → 1 (unequal)]
```

🔹 Topic 7: Probability Distributions

In Univariate Analysis, understanding how a single variable behaves **probabilistically** is crucial — that's where **Probability Distributions** come in.

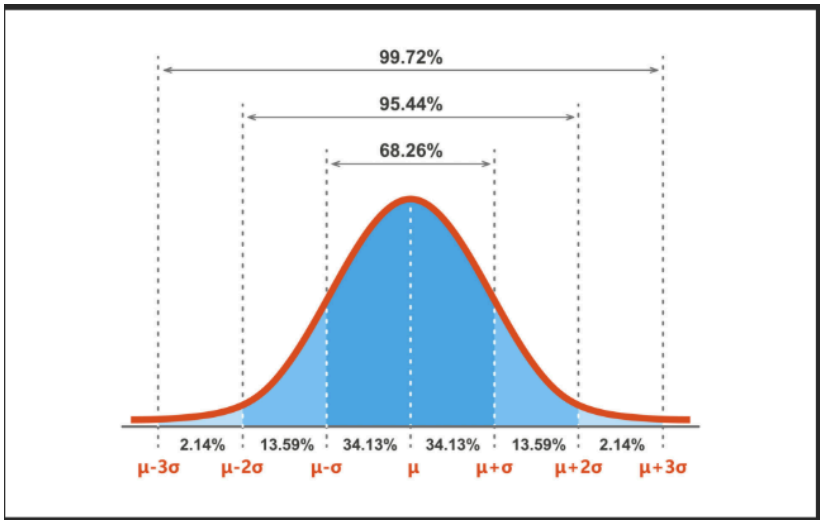
We'll cover:

- 📘 **What is a probability distribution?**
- 📊 **Gaussian (Normal) Distribution**
- 🎯 **Binomial Distribution**
- 🔢 **Poisson Distribution**

✅ What is a Probability Distribution?

- A **mathematical function** that tells us **how likely** values of a variable are.
- It shows **frequency or probability** of occurrence for each value.
- Types:
    - **Discrete**: e.g., Binomial, Poisson (countable outcomes)
    - **Continuous**: e.g., Gaussian (infinite possible values)

1️⃣ Gaussian (Normal) Distribution



📘 **Definition:**

- A **bell-shaped**, symmetric curve
- Most data clusters around the **mean**
- Used in **heights, IQ scores, measurement errors, stock returns**

📐 Formula:

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \cdot e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

Where:

- μ: Mean
- σ: Standard deviation
- x: Value

🔁 **Properties:**

Property	Description
Symmetric	Mean = Median = Mode
68-95-99.7 Rule	~68% in 1σ, 95% in 2σ, 99.7% in 3σ
Bell shape	Tails never touch x-axis

🐍 **Code + Plot:**

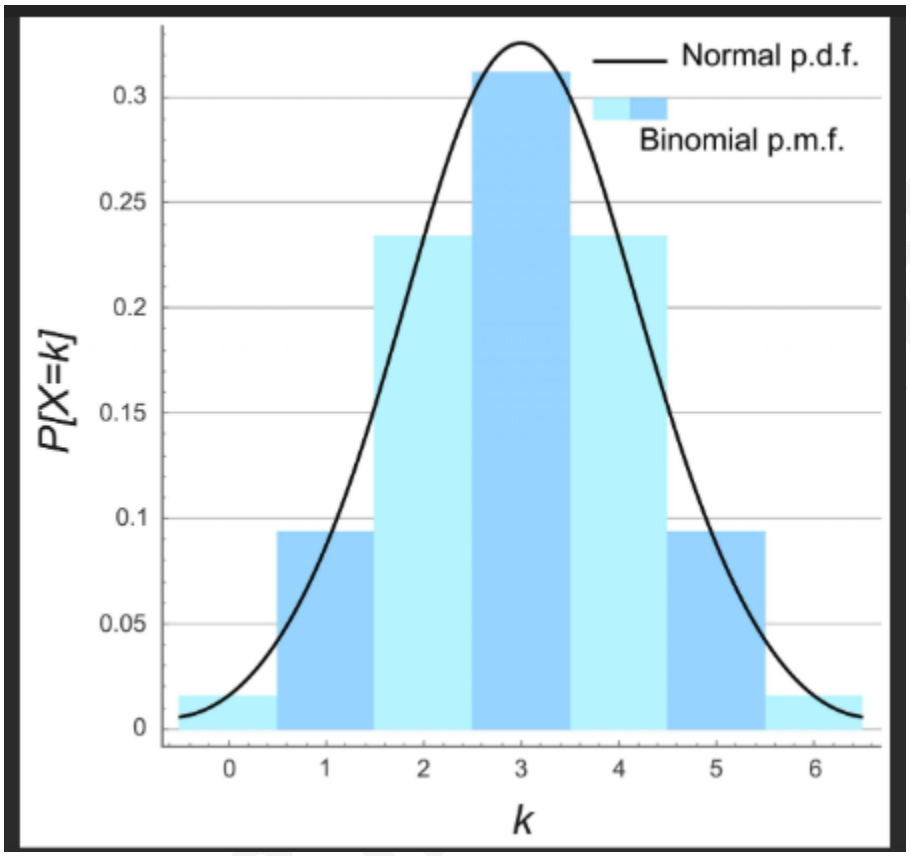
python
CopyEdit

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm

x = np.linspace(-5, 5, 1000)
mean = 0
std_dev = 1
pdf = norm.pdf(x, mean, std_dev)

plt.plot(x, pdf, label='Normal Distribution')
plt.title('Gaussian (Normal) Distribution')
plt.xlabel('x')
plt.ylabel('Probability Density')
plt.grid(True)
plt.legend()
plt.show()
```

✅ **Use Case**: Heights of students in a class, machine tolerance, stock prices

---

2️⃣ Binomial Distribution



📘 **Definition:**

- Describes the number of **successes** in a **fixed number of independent yes/no trials**
- Example: Tossing a coin 10 times and counting heads

📐 Formula:

$$P(X = k) = \binom{n}{k} p^k (1-p)^{n-k}$$

Where:

- n: number of trials
- k: number of successes
- p: probability of success in one trial

🔁 Properties:

Feature	Description
Discrete	Count of success
Range	0 to n
Shape	Symmetric if p = 0.5, skewed otherwise

🐍 Code + Plot:

python
CopyEdit

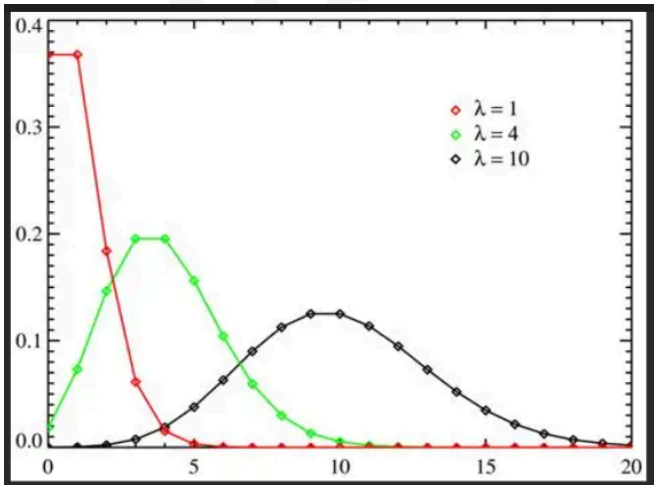
```
from scipy.stats import binom

n = 10   # trials
```

```
p = 0.5  # probability of success
x = np.arange(0, n+1)
pmf = binom.pmf(x, n, p)

plt.bar(x, pmf, color='skyblue')
plt.title('Binomial Distribution (n=10, p=0.5)')
plt.xlabel('Number of Successes')
plt.ylabel('Probability')
plt.grid(True)
plt.show()
```

✅ **Use Case**:

- Defective parts in a batch
- Number of students who pass an exam
- Response to marketing emails

3️⃣ Poisson Distribution



📘 **Definition:**

- Describes number of **events occurring in a fixed interval** of time or space, when they occur **independently**
- Example: Number of calls received at a call center per minute

📐 **Formula:**

$$P(X = k) = \frac{\lambda^k e^{-\lambda}}{k!}$$

Where:

- λ: Average number of events in interval
- k: Number of occurrences

🔁 **Properties:**

Feature	Description
Discrete	Event count
Mean = Variance	λ
Skewed	Skewed right for low λ

🐍 **Code + Plot:**

python

CopyEdit

```
from scipy.stats import poisson

λ = 4  # average events per interval
x = np.arange(0, 15)
pmf = poisson.pmf(x, μ=λ)

plt.bar(x, pmf, color='orange')
plt.title('Poisson Distribution (λ=4)')
plt.xlabel('Number of Events')
plt.ylabel('Probability')
plt.grid(True)
plt.show()
```

✅ **Use Case**:

- Number of emails per hour
- Website hits per minute
- Disease cases per day in a town

🔚 **Summary Table:**

Distribution	Type	Use Case	Shape
Gaussian	Continuous	Heights, test scores	Bell curve

Binomial	Discrete	Coin toss, pass/fail	Varies w/ p
Poisson	Discrete	Events in time/space	Skewed

🔄 Text Flowchart – How to Choose

scss
CopyEdit

```
Start
 |
 ├── Discrete Outcome?
 |     ├── Fixed trials (Y/N)? → Binomial
 |     └── Event count in interval? → Poisson
 |
 └── Continuous Outcome → Gaussian
```